Maximum Sum Submatrix

AlgoExpert · Hard · Dynamic Programming

The problem

- Largest sum among all size x size square submatrices, using 2D prefix sums.

Intuition

- Two-dimensional prefix sums give any rectangle's sum in constant time.

Brute force vs optimal

- Sum every k×k block directly → $O(R \cdot C \cdot k^2)$
- 2-D prefix sums → $O(R \cdot C)$
- Inclusion–exclusion per block

Approach

- 1. Build prefix sums
- 2. For each bottom-right corner, compute the size by size square sum
- 3. Track the maximum

Worked example

matrix

5	3	-1	5
-7	3	7	4
12	8	0	0
1	-8	-8	2

prefix sums

0	0	0	0	0
0	5	8	7	12
0	-2	4	10	19
0	10	24	30	39
0	11	17	15	26

Build 2-D prefix sums

Worked example

5	3	-1	5
-7	3	7	4
12	8	0	0
1	-8	-8	2

```
sum = P[r][c] - P[r-2][c] - P[r][c-2] + P[r-2][c-2] = 18
```

Best 2×2: 18

Java solution

```java
public class MaximumSumSubmatrix {
    public static int maximumSumSubmatrix(int[][] m, int size) {
        int R = m.length, C = m[0].length;
        int[][] p = new int[R + 1][C + 1];
        for (int r = 0; r < R; r++) for (int c = 0; c < C; c++) p[r + 1][c + 1] = m[r][c] + p[r][c + 1] + p[r + 1][c] - p[r][c];
        int best = Integer.MIN_VALUE;
        for (int r = size; r <= R; r++) for (int c = size; c <= C; c++)
            best = Math.max(best, p[r][c] - p[r - size][c] - p[r][c - size] + p[r - size][c - size]);
        return best;
    }
}
```

Complexity

- Time: O(w*h)
- Space: O(w*h)

Edge cases & pitfalls

- Add back the double-subtracted corner
- Off-by-one with padding
- Negative values

Interview tips & follow-ups

- Follow-up: max submatrix of any size (Kadane in 2-D)

Recap

- Two-dimensional prefix sums give any rectangle's sum in constant time.
- Complexity: O(w*h)